Phase 1 MVP - System Architecture Pack

AITrendNFTMarketplace

This document summarizes the finalized system architecture for Phase 1 (MVP): Ethereum-only ERC-721 minting, automated trend → AI generation → mint → listing pipeline, marketplace web app, admin dashboard, partner REST API, IPFS metadata, and payment gateway abstraction.

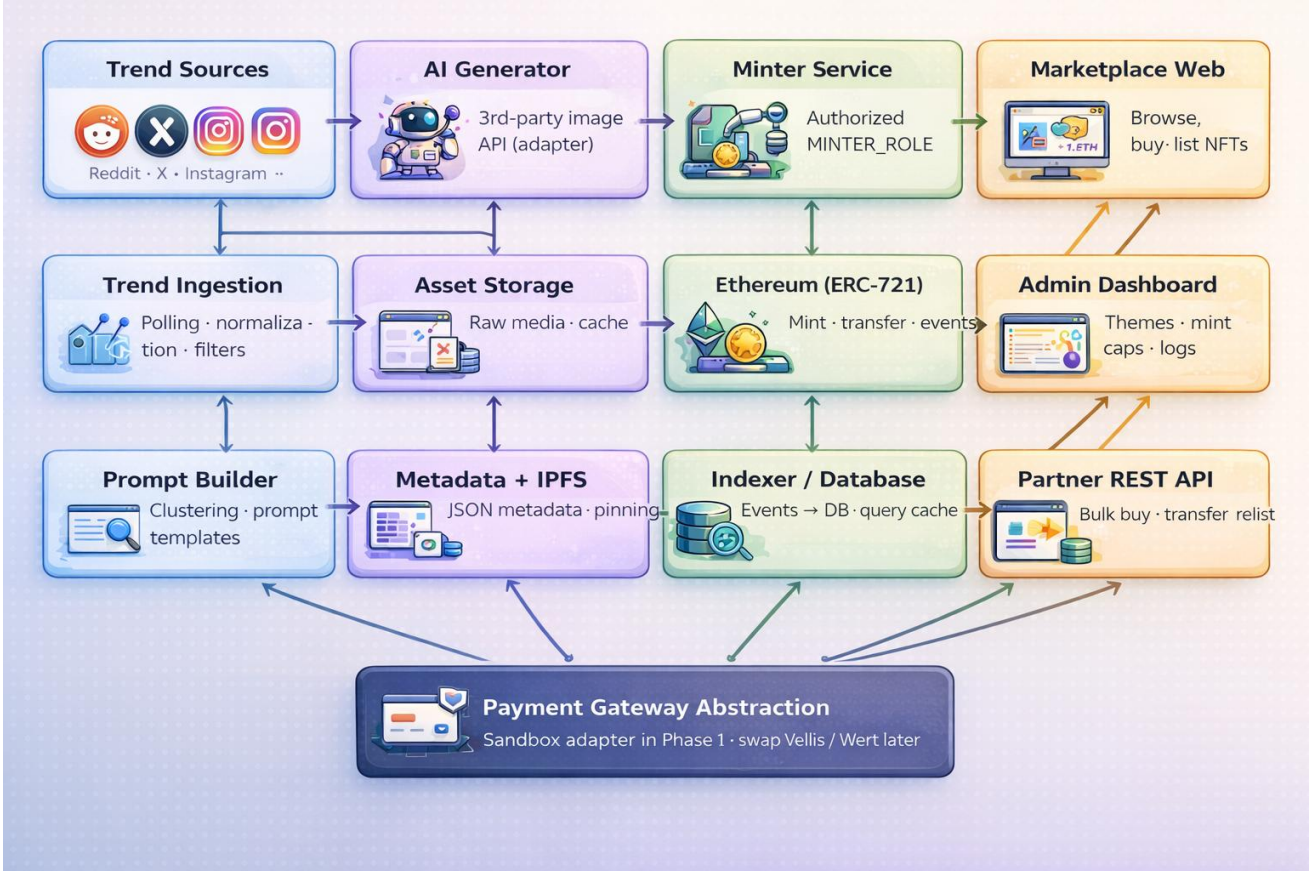
Version: 1.2 | Date: 2026-02-09

In scope (Phase 1)	Ethereum-only ERC-721; automated minting pipeline; marketplace web; admin dashboard; WalletConnect + MetaMask; REST API (bulk purchase/transfer/list); gateway abstraction (sandbox); account history & receipts; IPFS metadata; standard uptime with maintenance windows.
Out of scope (Phase 2)	Fractional ownership; advanced royalties; multi-chain; advanced analytics.

1. Architecture Overview

The platform is built as a set of modular services. Automation runs on schedules and event triggers: trend signals are ingested, converted into prompts, sent to a third-party AI generator, and the resulting assets are packaged into IPFS-hosted metadata. An authorized minter service mints ERC-721 tokens on Ethereum and immediately surfaces them in the marketplace via an indexer that listens to on-chain events.

High-Level Component Diagram



Design principles: (a) separation of on-chain vs off-chain logic, (b) gateway abstraction so payment providers can be swapped later, (c) backend-controlled minting via roles, (d) NFT metadata stored on IPFS.

2. Core Workflows

2.1 Automated Minting & Listing

- Scheduler triggers trend polling (configurable frequency and per-day mint cap).
- Trend Ingestion normalizes topics and applies filters (admin-pinned themes supported).
- Prompt Builder creates a provider-specific prompt payload (topic, style, constraints).
- AI Generator returns media assets; files are stored in asset storage and pinned as needed.
- Metadata Builder generates ERC-721 metadata JSON and pins to IPFS (ipfs://CID/... tokenURI).
- Minter Service (MINTER_ROLE) mints on Ethereum with tokenURI and edition settings.
- Indexer listens to on-chain events and updates the marketplace database so new NFTs appear withinminutes.

2.2 Purchase Flow (Web + Partner API)

- User connects wallet (WalletConnect/MetaMask) and selects an NFT to purchase.
- Marketplace requests a payment session via the Gateway Abstraction (sandbox adapter in Phase 1).
- When payment is confirmed/cleared, backend triggers the on-chain transfer (or listing settlement).
- Smart contract transfer finalizes ownership; indexer updates ownership state and receipts.
- User and admin can view transaction history and payment receipts.

3. Responsibilities & Interfaces

3.1 Component Responsibility Matrix

Component	Responsibilities	Primary Interfaces
Trend Ingestion	Poll trend sources; normalize; dedupe; apply rate-limit protections.	Scheduled jobs; internal events/queue
Prompt Builder	Transform topics into AI-provider prompt payloads; apply style templates.	Internal API
AI Generator Adapter	Call 3rd-party AI; handle retries/timeouts; store raw outputs.	AI provider API
Metadata + IPFS	Build metadata JSON; pin assets/JSON to IPFS; return ipfs:// URIs.	IPFS pinning service API
Minter Service	Hold MINTER_ROLE; mint ERC-721 with tokenURI; enforce edition rules.	Smart contract calls
Indexer/Listener	Listen to contract events; update DB; drive marketplace listings.	WebSocket/RPC; DB writes
Marketplace Web	Browse, buy, list; show owned NFTs; receipts UI.	Backend REST; wallet connection
Admin Dashboard	Manage sources/themes; mint controls; view logs/receipts.	Backend REST
Partner REST API	Bulk purchase/transfer/list endpoints; idempotency & auth.	REST/JSON
Gateway Abstraction	Unified payment interface; sandbox adapter; webhook ingestion.	Gateway API + webhooks

3.2 Key API Boundaries (Phase 1)

• Admin API: manage trend sources, pinned themes, generation caps, editions defaults.
• Marketplace API: listings, detail pages, ownership, receipts, purchase session creation.
• Partner API: bulk purchase, bulk transfer, create/list/relist; payment must clear before release.
• Webhook Listener: payment status updates from gateway adapters (sandbox in Phase 1).

4. Non-Functional Requirements & Extension Points

4.1 Security & Compliance (MVP)

• Backend-only minting via MINTER_ROLE; private keys stored in a secrets manager.
• Strict webhook verification for payment events; idempotent processing to avoid double-settlement.
• Rate-limit and retry logic for trend and AI provider calls.
• Audit-ready logs: mint events, payment events, transfer events, admin actions.

4.2 Scalability

• NFT output is throttled with configurable caps (per-hour/per-day) and queue-based processing.
• Stateless backend services can scale horizontally; storage/IPFS pinning can be scaled independently.
• Indexing uses on-chain events as the source of truth; DB is a cache for fast UI queries.

4.3 Phase 2 Hooks (No Rework)

• Royalties: add ERC-2981 or custom payout logic and integrate into sale settlement flow.
• Fractional ownership: introduce separate fractionalization contracts without changing ERC-721 basetokens.
• Multi-chain: abstract chain RPC + deploy pipeline; keep indexer pluggable per chain.
• Advanced analytics: expand event warehouse and reporting exports.

Sign-off

This architecture is the agreed Phase 1 baseline and will be used to guide implementation and testing across all milestones.